Lab Implementation Guide

How to use the guide

The analyst reads reports. The engineer reads maintenance instructions. The unassigned user is denied. Follow the assignment for required outcomes.

Follow the five phases in order: infrastructure, identity and endpoints, access controls, monitoring and recovery, then validation and submission. Read each procedure first. Obtain approval before each stage A-L, covering assets, changes, validation and rollback. Use the architecture and technical reference for detail.

Use private copies of the five Office files, numbered **01-05**. Record observations, dates and private evidence references in the named sections. Use synthetic users and separate lab passwords; keep raw evidence private. Leave unperformed checks **NOT TESTED**. If a result differs, stop the affected work, follow its recovery plan and log it in **Deliverable 04 > Issue Log**.

Phase 1: Prepared Infrastructure

Verify the Dell Precision 5820 with Proxmox, HP desktop with physical OPNsense, TP-Link TL-SG108E switch and GL.iNet Beryl AX upstream router.

Before you start: Have read-only stage A approval, the private network map and backup/recovery record, and Dell/HP console access. Missing map, version or recovery details mean stop. Changes need separate B (host), C (network) or D (firewall) approval; this phase installs nothing.

A. Verify the lab equipment and VM resources

WHERE

Dedicated lab administrator computer; Dell and HP consoles. Use private management addresses.

STEPS

1. Sign in at both consoles. Compare device identities and cables with the private map. Confirm the original Windows disk, data and services remain protected and their recovery record is available.
2. Open **Proxmox > Dell node > Summary**, then storage > **Summary**. Compare CPU, memory and usable storage with the capacity budget and reserves.
3. Select **DC1**, **WIN1** and **APP1 > Hardware** in turn. Compare processors, memory and disk allocations with that budget. Physical OPNsense on HP is outside these guest totals.
4. Compare the GUI inventory with device labels and the private physical/firmware record. Record a difference before continuing.

CHECK	Exactly three workload VMs; allocations total 6 vCPU / 16 GiB RAM / 160 GiB disks. Measured capacity also covers host, free-space and temporary-restore reserves. Both consoles and recovery records are available.

RECORD	**Deliverable 01 > Hardware & Readiness**, **Virtual Machines** and **Resource Allocation**. Enter observations and evidence references.

B. Verify network zones and management access

WHERE

Administrator computer: Proxmox, HP OPNsense and switch web pages.

STEPS

1. Open **OPNsense > Interfaces > Assignments**. Match Management, Corporate and Restricted to the HP ports and private map.
2. Open **Proxmox > Dell node > System > Network**. Compare bridge ports, addresses and VLAN-aware settings with the approved relationships.
3. Open each VM's **Hardware > Network Device**. Match bridge and VLAN tag to the map: DC1/WIN1 in Corporate; APP1 in Restricted, with one zone NIC each.
4. Open the switch's **VLAN > 802.1Q VLAN** and **802.1Q PVID Setting** pages. Compare tagged/untagged membership and port defaults with the map; change nothing.
5. Inspect the management restrictions: named administrator sources/services, effective host/device rules, supported multifactor authentication and console recovery.

> **CHECK** Three zones reach HP; no switch/host route bypasses OPNsense. Workload bridges have no host address. Only the dedicated path reaches management; APP1's approved overlay is the sole extra interface.

> **RECORD** **Deliverable 01 > Network Zones** and **Allowed Network Flows**.

C. Verify isolation and required network access

WHERE

Administrator computer: read-only Windows, Beryl AX and firewall/host views.

STEPS

1. Open Windows **Settings > Network & internet > Advanced network settings**. Compare adapters with the isolation record; flag extra Wi-Fi, VPN or shared connections.
2. In Beryl AX **INTERNET**, inspect **Ethernet**, **Repeater**, **Tethering** and **Cellular**. Keep upstream disconnected until independent isolation and egress are approved.
3. Compare FF-01 through FF-08 with effective firewall/host rules, states, NAT and listeners. Use the flow checklist; unlisted access stays denied.
4. Complete the IPv4/IPv6, route and trust checks. GUI inspection alone does not prove home/production isolation.
5. For separately approved active validation, use lab-owned mocks. Record allowed and denied behavior plus policy/log evidence under FWT-01/FWT-02. Never probe home or production.

> **CHECK** Map/flows match. FWR-01/FWR-02 need FWT-01/FWT-02 evidence and second observations. Unperformed active checks stay **NOT TESTED**.

> **RECORD** **Deliverable 01 > Allowed Network Flows** and **Deliverable 03 > Test Cases**.

Before Phase 2: Resolve mismatches; carry pending checks forward. For unexpected connectivity, stop and use console isolation/recovery.

Phase 2: Identity and Endpoint Security

Outcome: DC1 manages Windows lab accounts, WIN1 uses those accounts, and APP1 has a restricted Linux baseline. Approvals: E for the directory and F for endpoint/server settings. Tests: FWT-03/FWT-04; requirements: FWR-03/FWR-04. Have verified media, private addresses, recovery credentials and a lawful approved update/activation path before installation.

Step 7 - Create and install the three workload VMs

Purpose and location: in Proxmox **Create VM** and each guest's **Console**, create only the approved workloads. A virtual disk is a file-backed guest disk; do not pass through a physical Windows disk.

VM	Zone / bridge	vCPU	RAM MiB	Disk GiB
DC1	Corporate / vmbr10	2	6144	64
WIN1	Corporate / vmbr10	2	6144	64
APP1	Restricted / vmbr20	2	4096	32

1. For each VM choose the approved unused ID, name, storage and verified OS ISO. Set its allocation from the table. Give it one zone NIC and record its MAC. Check selected OS minimums against real host capability; pause for an approved adjustment if they do not fit.
2. For the conditional Windows 11 profile, select **System > BIOS: OVMF (UEFI)**, add an EFI disk with the approved Secure Boot keys and **TPM 2.0** storage. Use an approved supported CPU model. A compatible SATA disk/emulated NIC can avoid separate VirtIO drivers; if VirtIO is selected, attach only verified approved driver media. Never bypass setup requirements.
3. Start DC1 > **Console**. In Windows Server Setup choose the approved edition **with Desktop Experience**, **Custom installation**, and only the empty 64 GiB VM disk. Finish setup and set the local Administrator password privately. Remove the ISO after successful boot. Microsoft's installation guide distinguishes Desktop Experience from Server Core.
4. Install WIN1's approved domain-capable Windows edition to its own 64 GiB VM disk. Use the approved lab-only setup/activation method and the separate administrator's local setup identity; do not use a corporate/home account or a sign-in bypass. Missing lawful offline setup or safe connectivity is a blocker.
5. Install APP1 using the approved Linux ISO. For the Ubuntu example choose the normal server installation, the 32 GiB virtual disk and hostname **APP1**. Use the same separate privileged person's lab administration identity, not another project persona. Leave optional services, cloud identity integration and SSH off unless specifically approved. Use the Proxmox console for administration.
6. On Windows, **Server Manager > Local Server > Computer name** or **System Properties > Computer Name > Change** sets **DC1/WIN1** before domain work. Restart if asked. Record actual OS/build, media, resources, NIC and update/evaluation status in **01 / Virtual Machines**; do not describe an unactivated system as fully ready.

Check: four baseline VMs exist including OPNsense; each guest boots from its own virtual disk with the intended single bridge attachment. RAM/storage reserves still fit. Record selected profile and limitations in **02 / Implementation Overview**.

If it fails: use the VM console to check ISO, boot order, virtual disk/controller and supported hardware. Do not alter the host disk, add a VM or disable Windows security requirements to finish setup.

Step 8 - Give DC1 its lab address and install Active Directory

Purpose and location: on DC1 as local Administrator, create **ad.lab.test**, a separate Windows directory. Active Directory (AD) stores Windows users/groups; DNS resolves lab names. This creates a new forest, the top-level directory boundary, under E approval.

1. Open **Server Manager > Local Server > Ethernet > adapter Properties > Internet Protocol Version 4 > Properties**. Enter LAB_DC1_IP, the Corporate mask/prefix and LAB_FW_CORPORATE_IP as gateway. Use LAB_DC1_IP as the planned lab DNS server; it begins serving after promotion. Remove public/corporate alternate resolvers. Confirm the guest is DC1, its MAC matches vmbr10 and time/timezone are correct.
2. Open **Manage > Add Roles and Features > Role-based or feature-based installation**. Select DC1, check **Active Directory Domain Services**, accept its management tools and install.
3. Select the notification flag > **Promote this server to a domain controller**. Choose **Add a new forest**, enter **ad.lab.test**, retain **DNS Server** and the approved supported functional levels. Enter the Directory Services Restore Mode recovery password privately.
4. Review DNS options, the proposed NetBIOS name and database/log/SYSVOL paths. A missing parent delegation can be normal for this standalone .test forest; confirm that exact warning, rather than ignoring all prerequisite warnings. Install only after the prerequisite check is understood, then allow the approved restart. Microsoft's deployment procedure governs these choices.

DESTRUCTIVE / HIGH-RISK - DC1, elevated Windows PowerShell as local Administrator, E approval. This is an alternative to the role/promotion clicks above, not a second deployment. It installs AD tools and creates the same lab forest, prompting securely for recovery credentials and rebooting after promotion. No placeholder or inline password:

```
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName 'ad.lab.test' -InstallDNS
```

Check: after reboot, **Server Manager > Tools** includes AD Users and Computers, DNS and Group Policy Management. In DNS expand **DC1 > Forward Lookup Zones > ad.lab.test** and inspect DC1 and directory service records. A role-install success alone does not prove promotion. Record in **02 / Active Directory and User Roles**, with FWT-03 evidence in **03 / Test Cases**.

If it fails: inspect the reported prerequisite, DC1 name, static address, DNS and time. Do not rerun forest creation, suppress checks, demote DC1 or revert a DC snapshot casually. Directory recovery requires an explicitly approved supported plan.

Step 9 - Create the users, groups and DNS record

Purpose and location: in DC1 **Server Manager > Tools > Active Directory Users and Computers**, create three standard people and one separate administrator. A security group collects permissions; membership alone does not invent a permission.

1. Right-click **ad.lab.test > New > Organizational Unit** to create the approved containers for standard users, privileged identities and member computers. An organizational unit (OU) organizes objects and lets you link policy; it is not another network zone. Keep DC1 in **Domain Controllers**.

2. In the standard-user OU, select **New > User** for **ava**, **eli** and **uma**. Use those logon names, prompted unique private passwords and the approved password-change settings. In the privileged OU create only the separately named administrator. Secure built-in accounts separately; they are not extra project people.
3. Use **New > Group**, **Global** scope and **Security** type for **LabReaders**, **LabMaintainers** and **LabAdmins**. In **Properties > Members > Add**, enter names, **Check Names**, then save the membership below.

Person	AD group membership	Application role planned later
ava	LabReaders	Reports
eli	LabReaders and LabMaintainers	Reports and maintenance instructions
uma	Neither reader nor maintenance group	Neither page
Separate privileged administrator	LabAdmins	Infrastructure only; no fourth app account

4. Under the same approved rights plan, assign **LabAdmins** the required infrastructure rights. For WIN1 local administration, add that group to WIN1's local **Administrators** using the approved local/group-policy setting. Any DC Domain Admins membership must be explicitly covered by E approval; the custom LabAdmins name itself grants no rights. Check ava/eli/uma are absent from privileged built-in groups.
5. Open **Server Manager > Tools > DNS > DC1 > Forward Lookup Zones > ad.lab.test > New Host (A or AAAA)**. Enter **Name: app**, **IP address: LAB_APP1_IP**, replacing it privately. Add the IPv4 A record; do not invent an AAAA record or corporate forwarding. Check **app.ad.lab.test** resolves to the intended Restricted address. Microsoft documents DNS record management.

Check: inspect each group's Members tab and each user's Member Of tab. Exactly three standard users, three project role groups and one separate privileged administrator are recorded in **02 / Active Directory and User Roles**; built-in/service objects are secured but not counted as project personas.

If it fails: correct only the wrong entry under the approved change. Preserve the previous membership. Do not grant Domain Admins to a standard user to resolve a logon problem.

Step 10 - Join WIN1 and check Windows logon

Purpose and location: on WIN1 using its approved local administrator, join the separate lab directory so standard users can sign in and receive Windows policy.

1. In WIN1's adapter IPv4 properties, enter LAB_WIN1_IP, Corporate mask and LAB_FW_CORPORATE_IP. Set **Preferred DNS: LAB_DC1_IP**, with no external alternate. Ensure DC1 is running and time agrees.
2. Open **System Properties > Computer Name > Change**. Verify **WIN1**, select **Domain**, enter **ad.lab.test**, and provide the approved lab join credential through the prompt. On confirmation restart, following Microsoft's domain-join procedure.

3. On DC1, find WIN1 in AD Users and Computers and move it into the approved member-computer OU. On WIN1 choose **Other user**, sign in as **ava@ad.lab.test** and confirm the Windows session uses the lab domain. Test eli and uma logon only within the approved test scope.
4. As the lab administrator, open PowerShell for the bounded checks below. Record name resolution, standard logon and effective policy as separate observations.

READ-ONLY - WIN1, Windows PowerShell as the approved lab administrator; elevation for the secure-channel check. DNS queries must show the private DC1/APP1 addresses. The final command checks this member machine's domain relationship; it is not a DC1 health test. No replacements:

```
Resolve-DnsName dc1.ad.lab.test
Resolve-DnsName app.ad.lab.test
Test-ComputerSecureChannel -Verbose
```

Check: correct private answers and **True** for the secure-channel test, plus actual standard-user logon and policy. Record configuration in **02 / Active Directory and User Roles** and results in **03 / Test Cases**, FWT-03.

If it fails: inspect WIN1's DNS address, DNS records, time, domain name and DC1 host rules. WIN1/DC1 are on the same bridge; an OPNsense allow cannot fix a DC1 host denial. Do not add **-Repair**, rejoin repeatedly or use public DNS without a bounded correction approval.

Step 11 - Apply essential Windows policy and host rules

Purpose and location: under F approval, use DC1 **Group Policy Management** and Windows firewall/health views to protect DC1/WIN1. Group Policy distributes settings to the domain or selected OUs; a setting is useful only if it actually reaches its target.

1. In **Group Policy Management > Forest: ad.lab.test > Domains > ad.lab.test**, create/link the approved lab account-policy Group Policy Object (GPO) at the **domain root**. Edit **Computer Configuration > Policies > Windows Settings > Security Settings > Account Policies**. Enter the owner-approved minimum length/history/age/complexity and lockout threshold/duration/reset values from F approval. Do not invent those numeric values or link domain-account password policy only to a member OU.
2. Create/link the endpoint GPO to WIN1's computer OU. Under **Security Settings > Windows Defender Firewall with Advanced Security > Windows Defender Firewall Properties**, keep Domain/Private/Public profiles **On**, inbound **Block** and only reviewed exceptions. Keep Microsoft Defender protection enabled using **Computer Configuration > Policies > Administrative Templates > Windows Components > Microsoft Defender Antivirus**; do not enable a policy that turns antivirus off.
3. On DC1 and WIN1 open **Windows Defender Firewall with Advanced Security > Inbound Rules**. Inspect enabled built-in AD/DNS rules. In each required rule's **Properties > Scope > Remote IP address**, restrict it to the actual lab clients; check protocol/local service ports and enabled profiles too. If creating a missing rule, use **New Rule > Custom**, exact service/program, protocol/local ports, remote addresses, **Allow**, the reviewed profiles and a clear FF-02 name.

WIN1 to DC1 dependency	Destination port to review in DC1 host policy
DNS	TCP/UDP 53; APP1 may also use only DNS/time under FF-04

WIN1 to DC1 dependency	Destination port to review in DC1 host policy
Kerberos logon/password service	TCP/UDP 88 and 464
Directory lookup and policy files	TCP/UDP 389; TCP 445
RPC endpoint mapper and dynamic RPC	TCP 135 and the selected modern-server range, normally TCP 49152-65535
Lab time	UDP 123 only if that is the approved Windows time service
Global Catalog	TCP 3268 only if selected-version client behavior requires it

Use Microsoft's AD service table for the selected versions. Do not permit replication/trust/LDAPS services merely because they appear in the vendor's wider table. The dynamic range is a service dependency scoped to WIN1/DC1, not an all-host/all-port permit. Check overlapping default rules do not keep broad remote scope.

4. In the **WIN1 endpoint GPO** edit **Computer Configuration > Policies > Windows Settings > Security Settings > Advanced Audit Policy Configuration > Audit Policies > Logon/Logoff > Audit Logon**, enabling approved success/failure auditing. Create/link the separately approved **DC audit GPO** to the **Domain Controllers** OU, then edit its **Account Logon > Audit Kerberos Authentication Service** success/failure settings under that same advanced-audit path. WIN1's OU policy cannot apply to DC1. Include approved account-management auditing and ensure basic audit settings do not override advanced policy; inspect the effective advanced audit selection on each actual target before Phase 4.
5. On WIN1 use the local policy-result commands below. Check the intended GPOs are applied and WIN1 is in the correct OU; compare specific values with the local effective security-policy/firewall views. In **Windows Security**, inspect antivirus, firewall and update status. Log off/restart only when the approved setting needs it.
6. In **Computer Management > Local Users and Groups > Groups > Administrators**, confirm the three standard users are absent and the approved LabAdmins rights are present. Use the same read-only computer-policy query below as the harmless standard/admin privilege comparison; no configuration write is needed.

READ-ONLY - WIN1, first in ava's unelevated Windows PowerShell, then in the separate lab administrator's elevated shell. The first query shows ava's applied user policy. Run the second in each context: expect Access Denied as ava, then a computer-policy summary as the authorized administrator. No passwords/remote target arguments or placeholders. Microsoft documents gpresult:

```
gpresult /scope user /r
gpresult /scope computer /r
```

Check: the actual user/computer policy and privilege behavior agree with membership. An unexpected standard-user success requires investigation before PASS; a missing policy result requires checking OU/application/logon, not disabling protection. Use the full winning-setting view where the summary alone cannot prove a value.

Check: applied policy, enabled protection and scoped rules agree with standard logon, the standard-user denial and approved admin control. Record in **02 / Windows and Linux Endpoint Controls** and **03 / Test Cases**, FWT-04. Record update/activation gaps honestly.

If it fails: inspect OU link, security filtering, winning policy, profile and enabled rule scope. Restore only the changed pilot settings through retained administrator/console access. Never disable the firewall or antivirus as a shortcut.

Step 12 - Configure APP1 networking and its host firewall

Purpose and location: at APP1's Proxmox console under F approval, set a fixed Restricted address and permit only the application. The following is the conditional Ubuntu/Netplan/nftables profile. APP1 remains outside AD.

1. Match APP1's interface to its Proxmox MAC and inspect existing addresses/routes using the commands below. Save the current **/etc/netplan/** and firewall configuration into new approved private backup paths before editing; do not overwrite an earlier recovery copy. Record the actual renderer/backend in **02 / Windows and Linux Endpoint Controls**.

READ-ONLY - APP1 console, normal user for network views; sudo/root for protected firewall rules. These inspect addresses, routes, DNS and existing rules. No placeholders. Do not infer isolation from one view:

```
ip -br address
ip route
resolvectl status
sudo nft list ruleset
```

Check: a single zone NIC matches vmbr20, with no home/production resolver or unintended forwarding rules. If a tool/backend differs, stop and verify the selected profile rather than installing another manager.

2. Using the approved privileged editor, edit the one existing Netplan YAML that owns the NIC. Preserve indentation and remove conflicting definitions from the approved source only; a second YAML file can merge with old settings. Replace all capitals with approved private values. Ubuntu's network guide explains the fields.

CONFIGURATION CHANGE - APP1, root-owned /etc/netplan configuration, privileged editor. File content only; LAB_NIC, addresses, prefix and gateways come from the private Network Zones record. IPv6 routing/autoconfiguration is not enabled in this baseline:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    LAB_NIC:
      dhcp4: false
      dhcp6: false
      accept-ra: false
      addresses: [LAB_APP1_IP/LAB_PREFIX_LENGTH]
      routes:
        - to: default
          via: LAB_FW_RESTRICTED_IP
      nameservers:
        addresses: [LAB_DC1_IP]
        search: [ad.lab.test]
```

Check before applying: the address belongs to Restricted, the gateway is its OPNsense interface and DNS is DC1. Do not paste the placeholders literally. Preserve any release-specific required fields.

CONFIGURATION CHANGE - APP1 console, sudo/root, F approval. Generate checks the merged syntax; try temporarily applies it and asks for confirmation. No placeholders after the file is filled:

```
sudo netplan generate
sudo netplan try --timeout 120
```

Check: confirm only after the address/route/resolver views match and lab DNS works. Netplan warns that rollback must be checked; on error or timeout verify the actual state at console and restore the saved file/settings before continuing.

3. For the conditional Ubuntu **systemd-timesyncd** profile, preserve its current configuration. Confirm this service exists and no other time daemon/per-link DHCP NTP source overrides the plan; if a different daemon is selected, obtain its reviewed equivalent before continuing. FF-04 must permit the corresponding lab-only time dependency.

CONFIGURATION CHANGE - APP1 console, sudo/root, F approval. Create the selected service's configuration directory if absent; existing directory contents are preserved:

```
sudo install -d -o root -g root -m 0755 /etc/systemd/timesyncd.conf.d
```

Check: the directory is root-owned. In the approved editor save the following at **/etc/systemd/timesyncd.conf.d/10-lab-time.conf**, preserving an existing file before any edit.

CONFIGURATION CHANGE - APP1 root editor, 10-lab-time.conf. File content only. Replace LAB_TIME_IP with the exact approved lab time server; empty FallbackNTP removes the default fallback list:

```
[Time]
NTP=LAB_TIME_IP
FallbackNTP=
```

Check: only the selected lab server appears and permissions prevent ordinary-user edits. Do not use a public/corporate time source.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root for restart; conditional installed systemd-timesyncd only. Apply the reviewed time drop-in and inspect the selected source/clock. No placeholders after its NTP field is completed:

```
sudo systemctl restart systemd-timesyncd
timedatectl timesync-status
timedatectl status
```

Check: the intended lab source is selected, synchronization/timezone and observer UTC difference are recorded, and no fallback public server appears. If unavailable/unsynchronized, keep the dependency unresolved; do not add an Internet pool or change DC1 time. The systemd configuration reference describes global/per-link sources.

4. Prepare **/etc/nftables.conf** with the approved editor. The example below owns one fresh **inet lab_host** table; if that table already exists, review/reconcile it first instead of duplicating it. Preserve other tables. Do not use **flush ruleset**, which would erase other services' rules. Keep IP forwarding off and check both IPv4/IPv6 effective values at stage entry.

CONFIGURATION CHANGE - APP1, sudo/root editor, nftables only. Replace LAB_WIN1_IP and LAB_ADMIN_IP with exact approved IPv4 hosts. This table allows local HTTPS, restricts future overlay packets to HTTPS before established traffic, and drops forwarding. No SSH service is enabled by this example:

```
table inet lab_host {
    chain input {
        type filter hook input priority 10; policy drop;
        iifname "lo" accept
        ct state invalid drop
        iifname "tailscale0" tcp dport 443 counter accept
        iifname "tailscale0" counter drop
        ct state established,related accept
        ip saddr { LAB_WIN1_IP, LAB_ADMIN_IP } tcp dport 443 counter accept
    }
    chain forward {
        type filter hook forward priority 10; policy drop;
    }
}
```

Check: review the Ubuntu nftables behavior with the selected release: an accept in another base chain does not override a later drop here. The overlay rule still depends on Tailscale's exact source/destination grant in Phase 3. Approved egress remains restricted by OPNsense/physical isolation; this input example does not authorize Internet access.

CONFIGURATION CHANGE, preceded/followed by READ-ONLY - APP1 console, sudo/root. Check syntax first; only then load the new table and enable the package's persistent service. No replacements after the file is complete. Use this initial-load sequence once on the reviewed fresh table:

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
sudo systemctl enable nftables
sudo nft list table inet lab_host
```

Check: the input/forward drop policies and exact permits are active. Confirm persistence at an approved reboot, then test the HTTPS permit and denied listener once the application exists. If syntax/load fails, remain at console and restore the saved rules using the approved method; do not disable all filtering. Later edits need a reviewed table-replacement transaction, not repeated table creation.

For a later approved change to this existing table, preserve the active rules and boot file first. Edit the complete **/etc/nftables.conf** to include only the intended narrow change. Save the following separate root-owned transaction as **/root/lab-host-replace.nft**; it is for an already existing table, not the boot configuration.

CONFIGURATION CHANGE - APP1 root editor, /root/lab-host-replace.nft. This file replaces only lab_host in one atomic nftables transaction, preserving other tables. The included boot file must contain the complete reviewed lab_host definition:

```
delete table inet lab_host
include "/etc/nftables.conf"
```

Check: /etc/nftables.conf still contains table creation only, without an unconditional delete that would fail at a fresh boot. If lab_host is absent, use the initial-load path instead.

CONFIGURATION CHANGE preceded/followed by READ-ONLY - APP1 console, sudo/root, exact approved rule scope. nft -c checks without applying; continue only after it succeeds:

```
sudo nft -c -f /root/lab-host-replace.nft
sudo nft -f /root/lab-host-replace.nft
sudo nft list table inet lab_host
```

Check: the specific changed rule and existing drop policies agree with the saved reviewed file; retest affected permit/deny paths. On failure restore the saved complete table/boot file at console using the same reviewed transaction, never flush other tables. This replacement method also handles the temporary mock and restored-client permits later in the manual.

If it fails: stay at console, keep external access disconnected and restore the saved network/firewall files only through the approved method. Verify the actual rollback and source/route/rule state before resuming; do not stack another manager or disable the host firewall.

Phase 2 checkpoint and recovery

Compare the directory/group/policy configuration with actual standard WIN1 logon, DNS and harmless privilege checks (FWT-03/FWT-04). Record supported builds, updates, host firewalls and no bypass adapters. Restore only the approved changed memberships/policy/network settings at console. A directory rebuild or DC snapshot rollback needs separate supported recovery approval.

Phase 3: Zero Trust Access Controls

Outcome: one HTTPS application with two protected pages, six role decisions, application revocation and Tailscale access to that application only. Approvals: G certificate/trust, H application, I remote access. Tests: FWT-03/FWT-05/FWT-06; requirements: FWR-03/FWR-05/FWR-06.

HTTPS encrypts the browser connection and checks the server's identity. A certificate authority (CA) signs the application's public certificate. The CA's private key must remain encrypted and offline; only its public certificate is trusted by approved lab clients. A valid certificate secures the connection but does not give a user permission to read a page.

Step 13 - Create the offline CA and APP1 request

Purpose and location: under G approval, use the existing approved offline signing workstation for the CA and APP1's console for its server key. Do not create a CA VM or install certificate services.

1. On the offline signer, open a terminal in a new approved owner-only directory outside every Git checkout. Use a POSIX-compatible shell and OpenSSL 3.x matching the req reference. Inspect the directory first: the following output paths must not already contain keys, certificates or serial state.

CONFIGURATION CHANGE - offline signer, signing-file owner; root not required. Set private path variables in the already approved current directory and restrict new-file permissions. There are no secret values in these assignments:

```
umask 077
LabCaKeyPath="$PWD/lab-ca.key"
LabCaCertPath="$PWD/lab-ca.crt"
LabAppCsrPath="$PWD/app.csr"
LabAppCertPath="$PWD/app.crt"
LabLeafExtensionsPath="$PWD/app-extensions.cnf"
LabSerialPath="$PWD/lab-ca.srl"
```

Check: these six paths resolve inside the approved private signing directory; outputs are new and source transfers will be reviewed. If any path exists unexpectedly, stop rather than replace it.

CONFIGURATION CHANGE - same offline signer/file owner. Create one 3072-bit encrypted CA key and a 365-day public root certificate. Keep the prompted passphrase in approved recovery custody; never supply it on the command line:

```
openssl req -x509 -newkey rsa:3072 -sha256 -days 365 -keyout "$LabCaKeyPath" -out
"$LabCaCertPath" -subj "/CN=HL-CYB-001 Lab CA" -addext
"basicConstraints=critical,CA:TRUE" -addext "keyUsage=critical,keyCertSign,cRLSign"
```

Check: the key and public certificate are created without an error. Inspect the public properties and encrypted-key readability without printing the key:

READ-ONLY - offline signer, signing-file owner; no root. These show public CA constraints/fingerprint and check the encrypted key through its passphrase prompt. No replacement values beyond the defined paths:

```
openssl x509 -in "$LabCaCertPath" -noout -fingerprint -sha256 -ext
basicConstraints,keyUsage
openssl pkey -in "$LabCaKeyPath" -noout -check
```

Check: CA:TRUE, certificate-signing usage, a recorded SHA-256 fingerprint and successful key check after the passphrase prompt. Store only public properties in **02 / HTTPS Certificate Configuration**. Failed key/passphrase verification blocks signing; preserve files and resolve custody without regenerating over them.

2. On APP1, prepare **/etc/labapp** as a restricted directory for the application. In the conditional Ubuntu profile Apache workers use **www-data**; verify the selected package account first. Run these commands only for a new approved lab directory, not over existing owner content.

CONFIGURATION CHANGE - APP1 console, sudo/root, G/H approved scope. Create the private configuration directory and define new key/request paths in that root shell. The public/private filenames stay inside the lab layout:

```
sudo install -d -o root -g www-data -m 0750 /etc/labapp
sudo -i
umask 077
LabAppKeyPath=/etc/labapp/app.key
LabAppCsrPath=/etc/labapp/app.csr
```

Check: the directory's owner/group/mode match the approved Apache account and the two files do not already exist. If the service account differs, correct the reviewed profile first.

CONFIGURATION CHANGE - APP1, that root shell, new approved paths. Create its 2048-bit service key and certificate signing request (CSR). The request is public signing input; it is not yet a certificate:

```
openssl req -new -newkey rsa:2048 -noenc -keyout "$LabAppKeyPath" -out "$LabAppCsrPath"
-subj "/CN=app.ad.lab.test"
```

Check: request subject is app.ad.lab.test and the server key is root-owned/private. The **-noenc** option intentionally leaves this service key unencrypted so the service can start; strict permissions and encrypted backup protect it. Inspect the CSR with **openssl req -in "$LabAppCsrPath" -noout -subject -verify** in the same root shell: expect the matching subject and successful request signature. On error, stop before transfer and preserve the current files.

3. Transfer only **app.csr** to the signer's LabAppCsrPath through the approved private media path. Keep the APP1 key on APP1 and the CA key on the offline signer. Compare the request name before signing; never publish either private key or its passphrase.

If it fails: preserve existing keys/request and stop before signing or trust changes. Resolve file identity, permissions, passphrase custody or the selected OpenSSL syntax; never overwrite a key to hide the error.

Step 14 - Sign the certificate and prepare browser trust

Purpose and location: on the offline signer, issue one application certificate for app.ad.lab.test; later import only the public root on approved lab clients.

1. Use the approved editor to save the following as LabLeafExtensionsPath. Extensions describe what the certificate is allowed to identify: **SAN** is its subject alternative name, and **serverAuth** means a web-server certificate.

CONFIGURATION CHANGE - offline signer, signing-file owner; no root. Exact configuration-file content, not shell commands:

```
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:app.ad.lab.test
```

Check: the leaf is not a CA, permits server authentication and names exactly app.ad.lab.test. A different hostname needs review before signing.

CONFIGURATION CHANGE followed by READ-ONLY - offline signer, signing-file owner. Sign the reviewed CSR for 90 days, maintaining private LabSerialPath; then verify the chain/name and show dates/SAN. Variables are those defined in Step 13. OpenSSL x509 and verify document the options:

```
openssl x509 -req -in "$LabAppCsrPath" -CA "$LabCaCertPath" -CAkey "$LabCaKeyPath"
-CAcreateserial -CAserial "$LabSerialPath" -days 90 -sha256 -extfile
"$LabLeafExtensionsPath" -out "$LabAppCertPath"
openssl verify -CAfile "$LabCaCertPath" -verify_hostname app.ad.lab.test
"$LabAppCertPath"
openssl x509 -in "$LabAppCertPath" -noout -dates -ext subjectAltName
```

Check: verification returns **OK**, the DNS SAN matches, and current time lies within the displayed validity. In the same certificate view inspect CA:FALSE, serverAuth and key usage; calculate/record remaining validity from the displayed end date and actual clock. Do not change domain time to test expiry. Failure blocks service use; do not bypass it with browser warnings.

2. Return **app.crt** and the public **lab-ca.crt** to APP1 through reviewed media. Place them at **/etc/labapp/app.crt** and **/etc/labapp/lab-ca.crt** with root ownership and approved public-certificate read access. Keep **/etc/labapp/app.key** root-only, mode 0600. Inspect owners/modes with the file-properties view or root **stat**; never display key content.
3. Prepare the public root and signer fingerprint for WIN1, but **test the untrusted browser first after Step 16 starts HTTPS**. A private/incognito window does not remove machine-wide CA trust. Use this before/after sequence on the existing WIN1 client rather than adding a VM.

4. After the untrusted rejection is recorded, on the approved WIN1 lab client open **Manage computer certificates > Trusted Root Certification Authorities > Certificates > All Tasks > Import** as administrator. Import only the public lab-ca.crt, comparing its fingerprint with the offline signer's record. Computer-wide trust must be explicitly approved; never add it to a home/production store. Microsoft describes the root store.

Check: record the one CA/one leaf, public properties, trust scope, normal trusted handshake and untrusted-client failure in **02 / HTTPS Certificate Configuration** and **03 / Test Cases**, FWT-05. Certificate-revocation services are excluded; state that limitation. Account/device revocation remains required.

If it fails: inspect SAN, chain, dates, private DNS and the selected trust store. Remove only an incorrectly imported lab root under G recovery approval; never trust an unexpected certificate to finish a test.

Step 15 - Create the two pages and separate app accounts

Purpose and location: on APP1 under H approval, build two static pages. The application has **separate test accounts**, distinct passwords and **manually assigned roles**. AD handles Windows logon; these Apache accounts handle the web pages. There is no single sign-on or automatic AD synchronization.

1. Use the approved package/update path to make the selected Apache 2.4 package and password utility available. In the Ubuntu profile, the proposed package command below installs the existing approved application tools only. Use approved local media when offline; do not connect to the Internet merely to complete installation.

CONFIGURATION CHANGE - APP1 console, sudo/root, H approval and approved package source. For the selected Ubuntu profile, install Apache and its password utility; no placeholders:

```
sudo apt install apache2 apache2-utils
sudo install -d -o root -g root -m 0755 /srv/labapp/reports /srv/labapp/maintenance
```

Check: the package manager reports success and the two root-owned directories exist. If packages are unavailable or signatures fail, stop and resolve the approved media path; do not disable verification or add a repository ad hoc.

2. In the approved root editor, save a small **index.html** in each directory. For **/reports/index.html**, enter a title such as **Purple Eye - Synthetic Report** and a sentence identifying fictional data. For **/maintenance/index.html**, use **Purple Eye - Synthetic Maintenance Instructions** with harmless fictional instructions. Do not insert real business information. Pages should be root-owned, readable by Apache and unwritable by ordinary app users.
3. Check **/etc/labapp/users** does not exist before the first password command. **-c creates or overwrites the whole file**; use it once only. Prompt for distinct app passwords, never AD passwords. Apache documents htpasswd's overwrite and hashing behavior.

CONFIGURATION CHANGE - APP1 console, sudo/root; the first line is DESTRUCTIVE / HIGH-RISK if users already exists. Create three bcrypt-hashed application accounts interactively, then restrict the file to administrator and Apache reads. The usernames are fixed synthetic names:

```
sudo htpasswd -B -c /etc/labapp/users ava
sudo htpasswd -B /etc/labapp/users eli
sudo htpasswd -B /etc/labapp/users uma
sudo chown root:www-data /etc/labapp/users
sudo chmod 0640 /etc/labapp/users
```

Check: all three writes succeed; inspect account names/count without copying hashes into evidence. If a write fails, preserve the old file and resolve the cause. Do not rerun **-c** to add the next person.

4. Save the group file below at **/etc/labapp/groups** using the approved root editor. Set owner/group **root:www-data**, mode **0640**. LabAdmins is infrastructure-only and does not become a fourth application account or role.

CONFIGURATION CHANGE - APP1 root editor, /etc/labapp/groups. File content only; memberships mirror the approved AD roles manually:

```
LabReaders: ava eli
LabMaintainers: eli
```

Check: ava/eli can be readers, only eli a maintainer, and uma has neither role. Record the mapping/custody in **02 / Application Accounts and Permissions**, without passwords or hashes. A directory-group change alone will not update this file.

If it fails: correct the specific spelling/membership/permissions from the saved version. Do not replace group authorization with **Require valid-user**, which would allow every authenticated account.

Step 16 - Enable HTTPS and run the six application cases

Purpose and location: in APP1's Apache configuration, expose only the two HTTPS paths, with each path checking its own group. Use the conditional Ubuntu package layout; different selected paths need review first.

1. Save the current Apache configuration privately. In **/etc/apache2/ports.conf**, retain **Listen 443** once for the TLS module and remove the plain **Listen 80** line under H approval. Do not create a duplicate 443 listener. Disable only confirmed default site links, not unknown owner configuration.
2. Save the following as **/etc/apache2/sites-available/labapp.conf** in the root editor. Verify all referenced files exist and that Apache's privileged startup can read the server key.

CONFIGURATION CHANGE - APP1 root editor, Apache 2.4 site configuration. Exact file content; no shell commands. TLS (Transport Layer Security) protects HTTPS, while group checks decide page access:

```
<VirtualHost *:443>
    ServerName app.ad.lab.test
    DocumentRoot "/srv/labapp"
    SSLEngine on
    SSLCertificateFile "/etc/labapp/app.crt"
    SSLCertificateKeyFile "/etc/labapp/app.key"
    SSLProtocol -all +TLSv1.2 +TLSv1.3
    ErrorLog ${APACHE_LOG_DIR}/labapp-error.log
    CustomLog ${APACHE_LOG_DIR}/labapp-access.log combined
    <Directory "/srv/labapp">
        Options -Indexes
        AllowOverride None
        AuthType Basic
        AuthName "HL-CYB-001"
        AuthBasicProvider file
        AuthUserFile "/etc/labapp/users"
        AuthGroupFile "/etc/labapp/groups"
        AuthzSendForbiddenOnFailure On
        Require all denied
    </Directory>
    <Directory "/srv/labapp/reports">
        Require group LabReaders
    </Directory>
    <Directory "/srv/labapp/maintenance">
        Require group LabMaintainers
    </Directory>
</VirtualHost>
```

Check: certificate/key paths match the issued files, directory listing is off, and each child requires its intended group. Apache's default AuthMerging Off lets the child requirement replace the parent denial; inspect other inherited/global settings. mod_ssl defines the TLS directives. Do not log Authorization headers, passwords or tokens.

CONFIGURATION CHANGE - APP1 console, sudo/root, selected Ubuntu profile. Enable required modules/site and disable only the reviewed default sites. If a named default site is absent, inspect existing sites rather than disabling everything. No placeholders:

```
sudo a2enmod ssl auth_basic authn_file authz_groupfile
sudo a2dissite 000-default default-ssl
sudo a2ensite labapp
sudo apachectl configtest
```

Check: the configuration check reports **Syntax OK**. An error or unexplained warning blocks the reload; inspect the reported filename/line and restore the saved site if needed. Syntax success does not prove running HTTPS.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root. After Syntax OK, restart to apply the reviewed listener change and inspect service/listener state. No placeholders:

```
sudo systemctl restart apache2
sudo systemctl status apache2 --no-pager
sudo ss -lntp
```

Check: apache2 is active, TCP 443 listens and plain HTTP/extra application listeners do not. If it fails, inspect the service error and saved configuration; do not expose another port to make it work.

3. On WIN1, confirm app.ad.lab.test resolves correctly, then open **https://app.ad.lab.test/reports/** before importing the root. Expect an untrusted-issuer rejection; do not click through. Record this FWT-05 subcheck, then perform the approved public-root import in Step 14 and open the same name again. Expect normal HTTPS without a warning.
4. Use a separate clean browser context for each synthetic app account. Close all private windows between users so Basic authentication credentials do not carry into another case. Enter valid **application** credentials through the browser prompt, not AD passwords.

Case	App account	Page	Expected result
C01	ava	/reports/	Allow: synthetic report, normally HTTP 200
C02	ava	/maintenance/	Deny: normally HTTP 403
C03	eli	/reports/	Allow: synthetic report, normally HTTP 200
C04	eli	/maintenance/	Allow: instructions, normally HTTP 200
C05	uma	/reports/	Deny: normally HTTP 403
C06	uma	/maintenance/	Deny: normally HTTP 403

5. In the browser developer tools **Network** tab, reload the exact page and inspect the request status. Compare it with Apache's access/error logs and the group file. A wrong password/401 does not demonstrate authenticated role denial. Record the six outcomes once in **03 / Test Cases**, FWT-03.04-.09, and link them from **02 / Six Application Access Tests**. Keep a successful eli maintenance browser open for Step 17.

If it fails: check DNS/host rules first for connection failure; check chain/name/dates for HTTPS failure; check the authenticated user, exact group spelling and inherited authorization for a wrong decision. Restore the saved site, syntax-check and restart only under H recovery scope. Preserve failure evidence before fixing it.

Step 17 - Remove an application permission and prove it is gone

Purpose and location: on APP1 and DC1 under H/E scope, remove eli's maintenance role. This demonstrates revocation: access that worked before the change must be denied afterward.

1. Record the change time using the separate observer clock. Preserve the approved private group-file backup. In APP1 **/etc/labapp/groups**, change the maintenance line to **LabMaintainers:** with no member. Leave **LabReaders: ava eli** unchanged; save with the same restricted ownership/mode.
2. On DC1 **AD Users and Computers > LabMaintainers > Properties > Members**, remove **eli**. Record both changes. Apache reads its own group file; Windows logon/token refresh does not revoke an application role.
3. In the already authenticated eli browser, reload **/maintenance/** with cache disabled in the developer Network panel; ensure a fresh request went to the server. Then use a fresh browser context and valid eli app credentials for another maintenance request. Both must deny, normally 403. Reading an already downloaded page from cache is not a new authorization decision.

4. Compare both requests with the changed app group file and correlated Apache denial records. Record actual elapsed time, request times and any cache/session treatment in **03 / Test Cases**, FWT-03.10-.13. Summarize in **02 / Access Revocation**. Do not claim a timing target that the assignment does not set.

Check: the formerly allowed maintenance access is now denied in both contexts. Restore eli's membership only if the approved test/recovery scope explicitly permits it, and record the restoration; do not silently change expected outcomes for later tests.

If it fails: verify the edited file is the effective AuthGroupFile, the browser made a server request, inherited configuration is unchanged and no broad rule grants access. Keep the failure; do not delete the account or disable the whole site merely to produce a denial.

Step 18 - Enroll lab-only Tailscale with a narrow policy

Purpose and location: under I approval, use the separate lab Tailscale admin console, APP1 and one existing approved lab client. Tailscale forms an encrypted overlay network; its policy must permit only APP1 HTTPS. AD membership and app passwords do not grant overlay access.

1. Confirm **independently isolated** approved egress **before Internet connectivity**, lawful provider terms, separate lab identities/tailnet, supported MFA and recovery. A second NIC, Beryl AX/home-LAN connection or NAT is not proof of that boundary. If this dependency is missing, stop remote work and mark FWT-06 NOT TESTED.
2. In the lab admin console **Access controls/policy editor**, save the existing policy privately and replace permissive defaults with the explicit deny-all below **before enrollment**. A blank/omitted policy can retain permissive defaults. Review all grants, ACLs (access-control lists), SSH rules and tag owners; permissions are additive.

DESTRUCTIVE / HIGH-RISK - separate lab Tailscale policy editor, authorized tailnet administrator, I approval. Policy-file content, not commands. This explicit initial policy grants no network or SSH access:

```
{
  "acls": [],
  "grants": [],
  "ssh": []
}
```

Check: the vendor editor validates/saves it and there are no other allows. Tailscale's deny-all examples explain the explicit empty list. On validation failure retain the last reviewed safe policy and stop enrollment.

3. Use the approved verified selected-version packages from the official Linux installation instructions and Windows client installer. Do not pipe an unreviewed online script into a root shell. Installation/enrollment belongs to I approval and the approved egress path, not this documentation task.

DESTRUCTIVE / HIGH-RISK - APP1 console, sudo/root, I approval. Start the reviewed installed client and authenticate interactively into the separate lab tailnet. No key/token arguments; keep the displayed authentication URL private:

```
sudo tailscale up --accept-routes=false --accept-dns=false --ssh=false
```

Check: complete sign-in with the approved lab identity, compare the **Machines/Devices** entry to APP1, and keep subnet advertisements/exit-node/Funnel/Serve/sharing off. On mismatch stop and

remove the unapproved lab enrollment through the authorized recovery plan; never sign in with a production identity.

4. On the existing approved Windows remote client, use the Tailscale tray/menu **Log in** flow with its approved lab identity. Review its client preferences: no exit node, accepted subnet routes or corporate DNS configuration. Compare its Machines entry with the private device record. The remote client must have no direct physical/zone route to APP1 that could falsely make an overlay test pass.
5. Copy the two exact overlay IPv4 addresses into the private stage I record. In the editor replace the initial policy with the following conditional example, replacing both placeholders with those exact addresses. This scopes the enrolled client device as well as APP1; it does not grant every device of a user or an entire subnet.

CONFIGURATION CHANGE - separate lab Tailscale policy editor, authorized administrator. Replace LAB_REMOTE_OVERLAY_IP and LAB_APP_OVERLAY_IP privately; literal symbols are not a valid final policy. No wildcard source/destination or extra grant:

```
{
  "acls": [],
  "grants": [
    {
      "src": ["LAB_REMOTE_OVERLAY_IP"],
      "dst": ["LAB_APP_OVERLAY_IP"],
      "ip": ["tcp:443"]
    }
  ],
  "ssh": []
}
```

Check: the editor validates the exact grant syntax. Tie both addresses to approved device/identity records and inspect absence of additive paths; an address change requires review, never broadening to a range. No Proxmox, OPNsense, DC1, RDP or SSH resource is granted.

6. On the approved Windows remote client only, use the elevated approved text editor to add the private mapping **LAB_APP_OVERLAY_IP app.ad.lab.test** in **C:\Windows\System32\drivers\etc\hosts**, replacing the address privately. Preserve the original file, avoid duplicate mappings and keep local WIN1 using DC1's zone record. Verify the browser uses the overlay address; do not publish this file.
7. On APP1 compare the active nftables **lab_host** input/forward chains with Tailscale's effective rules. Tailscale manages its own netfilter rules, so an OPNsense permit/deny is insufficient. Keep the Step 12 independent drop table active and test it; do not select off/nodivert or add forwarding as a shortcut.

READ-ONLY - APP1 or the approved client, local terminal; standard operator where supported. Show current overlay state, with no replacements. The CLI reference documents this inspection:

```
tailscale status
```

Check: only the approved lab enrollment is involved; compare device state to the admin console and actual access checks. Output contains private addresses/identity data and stays private. Status alone is not proof of authorization.

If it fails: retain deny-all access and stop enrollment/use until exact lab identity, device, name mapping, egress and effective host policy agree. Recover only through the approved device/grant removal and saved private settings; do not broaden a rule to make status look connected.

Step 19 - Test and revoke remote access

Purpose and location: from the approved remote client under the frozen I/K test scope, prove that only the named application works and that removing authorization stops it. Reuse existing lab people/client; add no permanent device, user or VM.

1. On the approved remote client, trust only the public lab root under its separate approved trust scope. Open **https://app.ad.lab.test/reports/** with valid ava app credentials. Expect normal HTTPS and the report; confirm APP1's access log shows the overlay source, not a local zone bypass.
2. For the wrong-port test use Step 24's approved known listening APP1 mock port, with a successful administrator control first. Confirm the remote request is denied and inspect the vendor/host decision. A closed port does not prove a policy deny. Confirm no route/grant reaches other resources; do not send packets to home/production.
3. For the unauthorized-identity check, reuse the same approved lab client under the existing **uma** lab-only provider context if I approval covers that enrollment. Remove/sign out the approved enrollment first, enroll the unauthorized context and inspect its distinct device state/address. It must receive no allow. Record the deny and policy evidence; restore the approved context only through the approved sequence. A different browser app username alone does not change Tailscale identity.
4. With the approved remote context restored and HTTPS working again, record a change time. Remove its grant and, in **Machines/Devices > exact client menu > Remove**, revoke that specific lab device's authorization. Keep APP1 intact. Vendor removal guidance explains current behavior.
5. Retry an existing browser context with forced network reload and a fresh new request. Both must fail to reach APP1. Compare policy/device state and APP1/host observations; record the exact changes, cache/state treatment and actual time to denial. Re-enrollment requires approval and invalidates any assumption that the previous address remains authorized.

Check: 03 / Test Cases, FWT-06, contains permitted HTTPS, unauthorized identity, wrong listening-port/resource and existing/new revoked-request outcomes. Summarize in **02 / Tailscale Remote Access** and **Access Revocation**. These are separate from the six app-role cases.

If it fails: stop remote access, remove the grant/device under I recovery scope and inspect additive rules, actual source address, client bypass routes, host-chain counters and device state. Never broaden grants, advertise a subnet or add a second remote provider. Restore private name mapping/trust/egress only as authorized.

Phase 3 checkpoint and recovery

Match the six decisions and app revocation to the effective group file/logs (FWT-03), chain/name/dates and trusted/untrusted browser behavior (FWT-05), and remote vendor/host policy to permit/deny/revoke observations (FWT-06). Keep manual identity mapping and certificate-revocation limits explicit. Recovery uses saved app/policy files, lab-only trust removal and exact remote deauthorization; preserve failed evidence and key custody.

Phase 4: Monitoring, Automation and Recovery

Outcome: five explainable events, one denied-access investigation, one safely repeated capacity script and one isolated APP1 restore. Approvals: J and the explicit L automation/recovery portions. Tests: FWT-07/FWT-08/FWT-09; requirements: FWR-07/FWR-08/FWR-09. Use existing local logs; no collector, security-monitoring product or extra permanent VM is needed.

Step 20 - Open the logs and produce five benign events

Purpose and location: under J approval, view DC1 Security, OPNsense firewall and Apache access/error logs. These are three sources; Apache access/error files are one application source. Record source/observer UTC (Coordinated Universal Time), time differences, restricted access and rotation in **03 / Security Events** before comparing events.

1. On DC1 open **Event Viewer > Windows Logs > Security > Filter Current Log**. Select the relevant time range and event IDs **4768,4771**, after confirming Step 11's Kerberos success/failure auditing. These directory events are generated on the domain controller; a WIN1 interactive logon event is not automatically a DC1 event. Microsoft documents 4768 ticket requests and 4771 pre-authentication failures.
2. In OPNsense open **Firewall > Log Files > Live View**, filter the named lab interface/source/destination/service and confirm the intended deny rule is logged. Check current states if a repeated request produces no new rule match.
3. On APP1 read only the needed log interval using the following commands. Restrict exports to the approved log reader and keep originals private. Review **/etc/logrotate.d/apache2** and Windows log maximum-size/retention properties against the measured disk budget; change only approved values, without clearing logs.

READ-ONLY - APP1 console, sudo/root or approved log-reader access. Show the last 40 application records from the selected Ubuntu site paths. No replacements; output may contain private addresses/usernames:

```
sudo tail -n 40 /var/log/apache2/labapp-access.log
sudo tail -n 40 /var/log/apache2/labapp-error.log
```

Check: a request appears with the expected path/status/user/time and a denied request has its corresponding authorization context. If files are missing, inspect the active site's ErrorLog/CustomLog and Apache permissions; do not assume an empty view means no event occurred.

4. Produce the following categories once within the approved action/rate. Reuse an existing recorded app case or deny when its evidence meets the requirement; do not generate duplicate traffic just to fill a sheet.

Event	Approved action	Source and expected observation
E01 directory success	On connected WIN1, sign out/in as an existing standard lab user with valid credentials	DC1 Security: successful Kerberos request attributable to that online logon
E02 directory failure	Verify the selected synthetic account is not near lockout; make one deliberately wrong-password logon attempt, then stop	DC1 Security: the relevant failure, often 4771; no repeated guesses

Event	Approved action	Source and expected observation
E03 app allow	Request ava /reports/ with valid app credentials	Apache: authenticated allowed request, normally 200
E04 app denial	Request ava /maintenance/ with valid app credentials	Apache: authenticated role denial, normally 403
E05 firewall denial	Make Step 24's one approved request to a known listening lab-only mock through the intended deny	OPNsense: matching blocked flow/rule; a disconnected listener is insufficient

5. For each category record actual source/action, source UTC, observer UTC, time difference, concise event summary and private evidence reference in **03 / Security Events**. If using only two sources, obtain the approved selection that still makes all five categories observable. Do not count five identical web requests as five categories.

READ-ONLY - APP1 local Linux console, approved log-reader; elevate only if journal access requires it. Optional service troubleshooting shows system-journal entries since local midnight, without a pager; no placeholders:

```
journalctl --since today --no-pager
```

Check: match its timezone/service messages to the action; Apache requests remain in the separate files above. If permission/time filters hide entries, resolve the view before generating another event. systemd's journal reference explains these filters.

If it fails: check auditing, actual online rather than cached logon, time range/timezone, site log path, rule logging and rotation. Do not repeat failed passwords or disable lockout to manufacture a record. Preserve the missing-evidence result as NOT TESTED until resolved.

Step 21 - Explain one denied-access event

Purpose and location: in **03 / Denied Access Investigation**, explain an observed denial using the existing event, effective policy and a working allowed control. This is one investigation, not a new attack exercise.

1. Choose E04 or E05 and refer to its **Security Events** ID. Record who requested what, the machine/client context, time and actual visible outcome.
2. Check the intended control: for E04 compare authenticated app username, resource and group membership; for E05 compare source/destination/service and the exact OPNsense rule. Record the observed reason rather than guessing from the word denied.
3. Link a known working allowed control to the same service: for example eli's approved maintenance access before revocation, or the administrator's allowed mock connection. Confirm the target was listening when the denied request occurred.
4. Compare the log's time/status/rule with the browser/request observation and effective file/policy. Explain why a bad password, broken DNS or stopped service is not the intended authorization/segmentation result.
5. Record conclusion, second observation, evidence ID and any defect reference. Add an actual problem to **04 / Issue Log** if correction is needed; preserve the original failure and later retest.

Check: another reader can follow request, intended policy, observed deny and supporting control without seeing credentials or raw logs. Link FWT-07 in **03 / Test Cases**; do not claim broader detection or enterprise log-retention performance.

If it fails: keep the conclusion unverified and obtain the missing source/policy observation. Do not change an allow into a deny expectation after seeing the result.

Step 22 - Run the one capacity-summary automation

Purpose and location: under L automation approval, use local Windows PowerShell on **WIN1** as a standard user with access to the approved private output directory. The operation reads local disk capacity and replaces one JSON summary; it does not alter accounts, networking or system configuration.

1. Review the exact script below, its target allowlist and output location before saving it privately. Use an existing reviewed directory outside every Git checkout; inspect its permissions and ensure neither directory nor output is a junction/symlink pointing elsewhere. The path check below is not a substitute for this review.
2. Save the script in the approved private script file. Leave the WIN1/ad.lab.test checks unchanged in the real operation. Use the existing PowerShell policy/signing path; do not add an execution-policy bypass.

CONFIGURATION CHANGE - WIN1, local Windows PowerShell, standard user with approved write permission. This proposed script queries local fixed disks and writes only WIN1-capacity.json. OutputDirectory is the one required runtime path, not a credential:

```
param([Parameter(Mandatory)][string]$OutputDirectory)
$system = Get-CimInstance -ClassName Win32_ComputerSystem
if ($system.Name -ne 'WIN1' -or $system.Domain -ne 'ad.lab.test') {
    throw 'Target is outside the approved lab operation.'
}
$directory = Get-Item -LiteralPath $OutputDirectory -ErrorAction Stop
if (-not $directory.PSIsContainer) { throw 'Output must be a directory.' }
$cursor = $directory
while ($null -ne $cursor) {
    if (Test-Path -LiteralPath (Join-Path $cursor.FullName '.git')) {
        throw 'Keep the capacity record outside Git.'
    }
    $cursor = $cursor.Parent
}
$capacity = Get-CimInstance -ClassName Win32_LogicalDisk -Filter 'DriveType=3' |
    Select-Object DeviceID, Size, FreeSpace
$capacity | ConvertTo-Json |
    Set-Content -LiteralPath (Join-Path $directory.FullName 'WIN1-capacity.json')
-Encoding utf8
```

Check before running: expected target/domain and one private replacement file match the approval. If a path/target is uncertain, stop without running it.

3. Set **$LabCapacityScriptPath** and **$LabCapacityOutputDirectory** privately to the reviewed script file and output directory. Invoke it as follows; the call changes only the named summary.

CONFIGURATION CHANGE - WIN1, same standard-user PowerShell and approved paths. Run the reviewed local operation. Both variables must be assigned to approved absolute paths first; never use a remote target:

```
& $LabCapacityScriptPath -OutputDirectory $LabCapacityOutputDirectory
```

Check: WIN1-capacity.json contains **DeviceID**, **Size** and **FreeSpace**, in bytes. Compare each drive with **File Explorer > This PC > drive Properties**. Use the displayed byte counts or convert consistently; GiB means 1,073,741,824 bytes. A successful script exit alone is insufficient.

4. Run the same reviewed script a second time. Compare files changed and the current disk/settings views: only the intended summary is replaced; free space can change naturally. Record the observation in **03 / Automation Verification**, A01-A07, and reference FWT-08 in **Test Cases**.
5. For safe invalid-target testing, make a disposable private copy of the script and change only its expected hostname literal to an intentionally nonexistent lab name. Run that copy on WIN1 with the same reviewed output directory; expect the target error **before any output write**. Verify the existing summary is unchanged. Do not rename WIN1, change its domain or contact another computer. Preserve the reviewed real script unchanged.
6. For invalid-path testing, use the real script with an approved known nonexistent directory or an existing ordinary file as OutputDirectory. Expect the Get-Item/directory error and no new/replaced summary. Inspect the intended output separately after each rejection; remove the disposable test copy only under approved cleanup.

If it fails: stop and compare target/domain, write rights, literal path, units and script error. Do not elevate or widen paths to force success. Remove only the intended output if recovery permits; preserve failure evidence and the original script.

Step 23 - Back up APP1 and restore it in isolation

Purpose and location: under L recovery approval, use Proxmox Backup/Restore and an already approved encrypted external storage target. Restore one temporary APP1 copy while preserving the original and the only backup. A snapshot or ordinary Proxmox archive alone is not encrypted off-compute backup.

Before starting: the encryption/mount/transport backend is still an owner decision. L approval must identify the exact external device, its existing encryption, recovery-key custody, supported mount/unmount steps, storage capacity, unused temporary VM ID, cleanup and rollback. No disk formatting or new encryption product/server is prescribed here. If this prerequisite is missing, FWT-09 is NOT TESTED.

Before choosing the final backup state, decide under the existing E/H/I approval scope whether the lab should remain revoked or return to its normal analyst/engineer/remote workflows. For normal handover, restore eli to **both** AD/application LabMaintainers, and separately re-enroll/re-authorize only the exact approved remote client with the narrow grant. Record the changes, new device/address if applicable and affected permit/deny retests before backing up. If restoration is not approved, retain the revoked state and report the unavailable workflow as a limitation; do not claim normal engineer/remote acceptance.

1. On APP1 record the private backup manifest: synthetic pages under **/srv/labapp**, the active Apache configuration, **/etc/labapp** including hashed users/groups, server key/leaf and public root, and required guest networking/firewall/time settings. In the conditional Ubuntu profile use the commands below before shutdown. Do not copy keys/password hashes or the private manifest into public evidence.

CONFIGURATION CHANGE to one private manifest, otherwise READ-ONLY - original APP1 console, root shell, L approval. Inspect the named manifest path before creation; preserve any older record. The selected profile must have all listed paths. Hash declared files without printing their content:

```
sudo -i
umask 077
LabManifestPath=/root/APP1-manifest.sha256
find /srv/labapp /etc/apache2 /etc/labapp /etc/netplan /etc/nftables.conf
/etc/systemd/timesyncd.conf.d -type f -exec sha256sum -- {} + > "$LabManifestPath"
```

Check: no path/read errors, every declared file has a manifest entry and no file content was printed. Copy this private original manifest through the approved media path to separate protected custody before backup; it is the comparison reference, not a public artifact. If a path differs/missing, correct the approved scope/profile instead of silently excluding it.

2. At the Proxmox console verify the approved external encrypted volume is actually mounted and unlocked using its approved backend procedure. Inspect the exact device/mount and available space; an empty directory on the host's own disk is not the backup target. Verify the separate recovery key can unlock the volume under the approved check. Keep that key separate from the stored backup.
3. In **Datacenter > Storage**, select the reviewed existing backup storage. For the conditional new **Directory** entry use **Add > Directory**, **ID: lab-backup** only if that unused ID is approved, **Directory: the exact mounted encrypted external path**, **Content: VZDump backup file**, and the intended node only. Require a mounted target with the guard below; do not allow fallback writes to the host directory. The directory-storage manual explains the backend; encryption comes from the approved underlying volume, not a Backup now checkbox.

CONFIGURATION CHANGE followed by READ-ONLY - Proxmox console, root, approved lab-backup Directory entry only. Set LabBackupMountPath privately to that entry's exact external mount path first. The option marks storage offline when its mount is absent; it does not mount, format or encrypt a device. Verify syntax against the selected pvesm reference:

```
pvesm set lab-backup --is_mountpoint 1
findmnt --mountpoint "$LabBackupMountPath"
lsblk -o NAME,TYPE,FSTYPE,MOUNTPOINTS
pvesm status
```

Check: the exact path is a mounted external device, **lab-backup** is active with sufficient space and its storage configuration shows the guard. Compare the device/encryption chain and unlock/recovery result to the approved backend record; a filesystem label or active storage alone does not prove encryption. Missing/wrong mount or unverified encryption means stop before backup.

4. Cleanly shut down APP1 through its guest console and confirm **Stopped**. Keep it off for the drill. Select **APP1 > Backup > Backup now**, the verified encrypted external storage, **Mode: Stop**, and the approved compression. Review the selected VM/storage, then start. In the task log confirm successful completion and inspect archive identity, date, size and included disk(s). Record backup time/age and manifest in **03 / Backup & Restore**, B01-B03.

5. In the storage's **Backups** view select that exact APP1 archive > **Restore**. Enter the approved unused temporary ID and reviewed guest-disk storage. Keep **Start after restore/live restore** off; review the original ID is not selected. Start the independent observer clock **immediately before confirming Restore**. Restore without overwriting the original or sole archive. Proxmox's backup/restore manual governs selected-version details.
6. Before starting the copy, open its **Hardware > Network Device > Edit > Advanced > Disconnect** for every NIC. Set **Options > Start at boot: No**. Confirm original APP1 is still off. Start only the copy via **Console** and keep it disconnected.

CONFIGURATION CHANGE - restored APP1 copy's console only, sudo/root, L approval. Stop and disable the copied overlay service before any client attachment. Do not run this on the original APP1. Conditional Ubuntu/systemd profile; no placeholders:

```
sudo systemctl disable --now tailscaled
sudo systemctl is-active tailscaled
```

Check: the copied service is inactive; the second command normally exits nonzero for inactive. Inspect the copy's NIC-disconnected state independently in Proxmox. If service identity/state is unclear, leave the copy disconnected and stop the drill.

Before changing the copy's firewall for testing, transfer the separately retained private original manifest to its approved comparison path and check restored integrity:

READ-ONLY - restored APP1 copy console, root for protected files. Set LabOriginalManifestPath privately to the approved original manifest file, then compare all declared files. No file is modified:

```
sha256sum -c "$LabOriginalManifestPath"
```

Check: every declared file reports **OK** before temporary test changes. If a file is missing/mismatched, preserve that result and investigate the archive/scope; do not rewrite the manifest to match the restore.

7. Use the approved existing Restricted bridge for the temporary recovery test **only after disconnecting every other path**: original APP1 stays off; disconnect OPNsense's vmbr20 NIC; attach the one existing WIN1 test NIC to vmbr20 with a temporary unused Restricted address, no gateway, and a private hosts entry for app.ad.lab.test. Preserve WIN1's previous adapter/hosts/bridge settings first. Confirm no physical port, host address or other route exists on vmbr20. Do not create a fourth permanent zone or attach the copy to Management/home.
8. While still disconnected, save the restored copy's original firewall file. On that **copy only**, use Step 12's scoped replacement transaction to add **ip saddr LAB_RESTORE_CLIENT_IP tcp dport 443 counter accept**, substituting the approved temporary WIN1 address. Put it after the existing physical-source HTTPS permit and before the input chain's closing brace; retain the earlier tailscale0 drop and final drop policy. Record this intentional post-integrity-check difference.
9. Attach only the copy's single NIC to isolated vmbr20 and enable its link. Confirm no live duplicate or Tailscale service. Reuse WIN1's approved lab root trust and map app.ad.lab.test privately to the restored address. Restore/remove the temporary permit during cleanup; do not broaden the source subnet.

10. Stop the independent observer clock when the declared integrity/service checks establish usable application recovery. Open HTTPS and test **ava /reports/** allow and **ava /maintenance/** authenticated deny, which hold in both approved eli membership states. Include this functional verification in actual elapsed time. Syntax checks alone do not establish recovery. Record time, integrity and behavior in **03 / Backup & Restore**, B04-B10, and FWT-09 in **Test Cases**.
11. Shut down/disconnect the copy after review. Under approved cleanup restore WIN1's saved NIC/address/hosts settings and the firewall NIC attachment, verify the copy cannot start/connect automatically, then resume original APP1 only after the isolation check. Verify original HTTPS/roles and overlay state through their approved path. Keep the sole backup safe; any copy deletion needs the exact cleanup scope.
12. Safely unmount/lock and detach the external encrypted backup through the approved backend procedure and store it away from the compute host. Record readable recovery custody, backup age, measured recovery and exclusions. This drill does not test AD, CA or Proxmox disaster recovery and promises no enterprise recovery-time target.

Check: a real backup/restore task, independent clock, manifest comparison and restored HTTPS/role behavior agree. The final off-compute copy is encrypted, with separate recovery access. No original disk, original APP1, sole backup or duplicate overlay identity was overwritten/connected.

If it fails: keep original and backup safe, leave the copy disconnected, record the failure and inspect encryption access, archive/disk coverage, storage space, network/firewall and service state. Do not reconnect broadly, retry against the original ID or erase the only archive.

Phase 4 checkpoint and recovery

FWT-07 requires five categories plus one explained denial; FWT-08 requires actual output, safe repeat and target/path rejection; FWT-09 requires an actual isolated restore with data/HTTPS/role evidence. Record only observed results. Recovery preserves source logs and backups, changes only approved settings/output, and leaves the recovery copy disconnected until exact cleanup is approved.

Phase 5: Validation and Final Submission

Outcome: ten fully evidenced engineering results, five clear submissions and a separate owner acceptance decision. Approvals: K validation and remaining L closeout. Test: FWT-10; requirement: FWR-10. Earlier observations can be reused only while they describe the current configuration and meet every mandatory check.

Step 24 - Prepare the permitted and denied network checks

Purpose and location: under the exact D/H/I/K validation scope, use existing lab guests and listeners to distinguish a real deny from a broken service. These are subchecks of FWT-01/FWT-02/FWT-06, not extra exercises or permanent resources. Freeze source, target, identity, single destination port, expected result, observer, stop condition and cleanup before each check.

1. Establish a working allowed control: WIN1 opens APP1 HTTPS and the dedicated administrator opens the lab management interfaces. Compare the active listener to the relevant permit/policy. If the control fails, diagnose it before the negative test.

2. For FWT-01's protected-boundary mock, use **APP1's lab address**, never a real protected address. Under the approved temporary rule scope, add a separate **MOCK_PROTECTED** alias containing that lab address and a logged Corporate rule blocking **WIN1 > MOCK_PROTECTED > TCP 443**, ahead of FF-03. Keep real protected networks physically absent. Make the one frozen request, compare its denial with this exact rule/counter, remove only the temporary mock rule/alias, and confirm normal FF-03 access again. The real isolation conclusion also requires cable/routes/trust review; the mock is one behavioral observation.
3. For FWT-02 unlisted inter-zone access, use DC1 as the source to the known listening APP1 TCP 443 while only WIN1 has FF-03 permission. For ordinary-user management denial, use WIN1 to Proxmox TCP 8006 with the dedicated-admin control working. Compare the intended network/host deny, not just a failed password prompt.
4. For FWT-06's wrong listening port, temporarily configure Apache's existing APP1 service to listen on one approved unused mock port such as **8443**. This example is conditional on that exact port/scope approval. Save the site/ports files, add the following separate temporary file in the approved Ubuntu Apache configuration directory, enable it, syntax-check and reload as in Step 16. No new page, account or software is needed.

CONFIGURATION CHANGE - APP1 root editor, temporary /etc/apache2/conf-available/lab-mock.conf; approved 8443 mock only. Configuration content, not commands; reuse the synthetic document tree and deny page content. The positive control needs only a listening TCP connection:

```
Listen 8443
<VirtualHost *:8443>
    ServerName app.ad.lab.test
    DocumentRoot "/srv/labapp"
    <Directory "/srv/labapp">
        Require all denied
    </Directory>
</VirtualHost>
```

Check before enabling: the port is unused, the exact temporary listener has approval, and no new Tailscale grant or ordinary-user permit is added.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root, temporary mock scope. Enable only that named configuration and verify syntax/listening. No placeholders once 8443 is approved:

```
sudo a2enconf lab-mock
sudo apachectl configtest
sudo systemctl reload apache2
sudo ss -lntp
```

Check: proceed to reload only after Syntax OK. Confirm Apache listens on 8443; if parsing fails, disable the named mock configuration and restore the saved files before reloading.

5. Use Step 12's console-only replacement transaction to add **ip saddr LAB_ADMIN_IP tcp dport 8443 counter accept** after the physical-source HTTPS permit and before the input chain's closing brace; retain the tailscale0 drop and final drop policy. Replace the source with the exact dedicated administrator IPv4 address. OPNsense's corresponding temporary FF-05 Management permit must name the same source/destination/port. Keep the vendor TCP-443-only grant unchanged. Confirm the administrator reaches the listener, then make the frozen remote wrong-port request; it must be denied. Match the actual vendor/host rule state/counter; do not claim one layer caused the deny without evidence.

6. After the recorded check, disable only **lab-mock** with the Ubuntu **a2disconf lab-mock** service helper, syntax-check, reload and confirm 8443 is no longer listening. Remove the exact temporary host/firewall permits through the saved transaction. Recheck production/home isolation and normal APP1 HTTPS. Preserve the temporary-rule evidence privately; add no permanent fifth VM or fourth zone.

READ-ONLY bounded connection check - approved Windows lab source, local PowerShell, standard user. Set **$LabApprovedTarget** privately to the one approved lab-owned address/name and **$LabApprovedPort** to its single frozen port. This makes a TCP connection attempt only; it is not a scan or a protected-network probe:

```
$connection = [System.Net.Sockets.TcpClient]::new()
try {
    $attempt = $connection.ConnectAsync($LabApprovedTarget, $LabApprovedPort)
    if (-not $attempt.Wait(5000)) { throw 'Lab connection timed out.' }
    'TCP CONNECTED'
} catch {
    'NO TCP CONNECTION: ' + $_.Exception.Message
} finally {
    $connection.Dispose()
}
```

Check: the positive control prints TCP CONNECTED; a denied attempt prints NO TCP CONNECTION and has matching intended policy/log/counter evidence. A timeout alone remains inconclusive. Do not supply a home/production address, loop through ports or rerun repeatedly to force a result.

If it fails: preserve the observation, restore only the named temporary configuration under the approved plan and recheck normal access. Missing approval or working listener means NOT TESTED, not permission to broaden access.

Step 25 - Execute and record the ten tests

Purpose and location: in **03-Security-Monitoring-and-Testing.xlsx / Test Cases**, turn observed behavior into an accurate engineering record. The workbook already contains the approved methods and mandatory child checks; do not create another result template in this guide.

1. Open **Test Cases**, enable the existing table's filter and select a **Test ID**. Read its **OVERALL** row and every numbered child row, along with the authoritative Test Plan. Freeze actual source/target/user/version, approval/date, action/rate, stop/rollback and evidence ID in the linked private record.
2. Perform the approved action, compare the outcome with the expected result and obtain the separate observation described below. Enter actual result, evidence, operator/date/version and defect/retest references in the existing columns. Never enter a planned outcome as an actual result.

Test / requirement	What to do and compare
FWT-01 / FWR-01	Steps 1-6/24: compare physical/firmware inventory to readiness records; inspect cables, bridges, other adapters, routes/IPv6, trust/data/credential isolation and disconnected or approved independent egress; prove the listening mock deny with its matching policy evidence.

Test / requirement	What to do and compare
FWT-02 / FWR-02	Count one host/four VMs/three zones and compare measured capacity/reserves. Verify FF-01-FF-08, named admin/MFA limits, console recovery, HTTPS permit, unlisted/ordinary-user management denial and same-zone host rules against actual sessions/logs.
FWT-03 / FWR-03	Steps 8-10/15-17: inspect directory/DNS/policy/counts and standard logon; run C01-C06 with valid app credentials. Compare app file/log decisions, then prove removed eli maintenance permission denies existing/fresh requests and record actual timing.
FWT-04 / FWR-04	Steps 11-12: inspect supported builds, updates, antivirus, applied policy, host firewalls, rights, single NICs/APP1 overlay exception, listeners/forwarding. Compare a harmless standard-user admin denial with the approved admin control.
FWT-05 / FWR-05	Steps 13-16: inspect encrypted offline CA custody and leaf chain/SAN/serverAuth/constraints/dates/remaining validity. Compare the normal trusted browser session with the otherwise working client's pre-import rejection. No warning bypass or domain-time change.
FWT-06 / FWR-06	Steps 18-19/24: inspect separate tailnet/MFA/recovery/device identity, narrow additive policy, no routes/shares/SSH/Funnel/Serve and effective host/DNS rules. Demonstrate approved HTTPS, unauthorized identity, wrong listening-port/resource and existing/new revoked access, with actual times/state treatment.
FWT-07 / FWR-07	Steps 20-21: match all five event categories to real actions, source/observer times and separate policy/group observations; inspect access/rotation and explain the one denial. Keep private source records.
FWT-08 / FWR-08	Step 22: review target/private-path guard, run the one operation, compare output with OS storage, repeat and check target/path rejection. Compare before/after state; only the intended summary may change.
FWT-09 / FWR-09	Step 23: inspect encrypted off-compute backup scope/key access/time/manifest, original-off/disconnected-copy state and inactive copied overlay. Compare restore output to independent time, hashes/content and actual isolated HTTPS allow/deny; state exclusions.
FWT-10 / FWR-10	Steps 25-27: check every requirement/mandatory result/evidence/defect, five files and unchanged rubric; complete sensitive-data/publication review and engineering cutoff/verifier. Verify actual publication through the linked governance workflow and keep owner UAT separate.

3. Mark **PASS** only when the actual result and all mandatory evidence/second observations match. Mark an observed mismatch **FAIL**. Mark an unexecuted, blocked or unverified mandatory check **NOT TESTED**. A mandatory test cannot be waived with NOT APPLICABLE.
4. Set the OVERALL row to PASS only after every mandatory child passes. Preserve failures and use **04 / Issue Log** for correction/retest. Do not change an expected deny into an allow or silently replace the original evidence.
5. In **03 / Requirements Coverage**, connect all ten FWR rows to the final test/evidence/defect references. Use **Evidence Index** for private-original references and separately reviewed public artifacts; raw paths/addresses never become public links.

Check: the ten OVERALL rows and every mandatory child have an accurate verdict and traceable evidence. A second observation can be another log/UI/policy view; it is not independent human

sign-off. Any missing mandatory evidence blocks engineering completion, and a critical safety failure blocks acceptance regardless of score.

If it fails: leave the affected result FAIL or NOT TESTED, explain the issue and obtain bounded remediation/retest approval. Continue only work whose dependencies remain safe; do not erase the failed attempt.

Step 26 - Finish the five Office deliverables

Purpose and location: in the private working copies of the five released templates, assemble the existing records into a clear submission. Keep detailed test execution in 03 and cross-reference it elsewhere.

1. **01-Lab-Environment-and-Architecture.xlsx:** finish **Hardware & Readiness**, **Virtual Machines**, **Network Zones**, **Allowed Network Flows** and **Resource Allocation**. Replace planned assumptions with observed fields only after verification, preserve procurement versus installation status and reference the private topology/public sanitized diagram appropriately.
2. **02-Identity-and-Zero-Trust-Implementation.docx:** complete **Implementation Overview**, **Active Directory and User Roles**, **Windows and Linux Endpoint Controls**, **Application Accounts and Permissions**, **HTTPS Certificate Configuration**, **Six Application Access Tests**, **Tailscale Remote Access**, **Access Revocation**, and **Implementation Results and Limitations**. Summarize configuration and link execution evidence in 03; do not paste password files/keys.
3. **03-Security-Monitoring-and-Testing.xlsx:** complete **Security Events**, **Denied Access Investigation**, **Test Cases**, **Requirements Coverage**, **Automation Verification**, **Backup & Restore**, and **Evidence Index**. Leave **Owner Acceptance** PENDING until explicit owner confirmation; test PASS does not fill that decision.
4. **04-Troubleshooting-and-Lessons.xlsx / Issue Log:** record actual problems, impact/cause, approved correction, verification/retest and practical lesson. Preserve unresolved items and original failure references; do not invent a troubleshooting story.
5. **05-Final-Project-Submission.docx:** complete **Executive Summary**, **Project Objectives**, **Architecture Overview**, **Implementation Summary**, **Security Testing Results**, **Key Findings**, **Challenges and Lessons Learned**, **Project Limitations**, **Final Evaluation**, and **Supporting Deliverables and Evidence**. Explain what works and what remains blocked, citing the owning records instead of copying their tables.
6. In **05 / Final Evaluation**, apply only the assignment's existing 100-point rubric: 25 infrastructure/network, 25 identity/access, 20 monitoring/operations, 20 testing/recovery and 10 documentation/submission. Record actual evaluator/date, engineering verifier and evidence cutoff. Unsupported plans earn no implementation credit.

Check: exact filenames and worksheet/section names match the blank templates, all claims point to observed evidence, limits are visible and raw private records remain outside Git. The separate project-level final report remains a template until its own real evidence/reporting is authorized.

If it fails: fix the editable working record and trace its claim to current evidence. A blank template, screenshot placeholder or total score cannot replace a mandatory test or owner acceptance.

Step 27 - Review evidence, submit and obtain owner acceptance

Purpose and location: at the approved publication workstation under L closeout scope, release only reviewed public-safe work and ask the owner to assess the intended workflows. Follow the engineering publication/build instructions and repository workflow; internal automation/branch mechanics belong there.

1. Review the whole proposed submission, not only the changed paragraph: filenames, document properties, hidden sheets/comments, screenshots, links and proposed Git history. Keep tokens/passwords, keys, hashes of passwords, actual addresses, private inventory, corporate details, raw logs/captures, backups and filled private working files outside the public set. Publish only explicitly reviewed sanitized summaries/artifacts. An ignore rule does not remove previously tracked content.
2. Capture evidence narrowly: show the relevant action/result and sanitized rule/group context. Keep source timestamps and private originals intact; review a separate redacted copy, including metadata. Do not alter the original to make a result look successful. Record reviewer/date and redaction/limitations in **03 / Evidence Index**.
3. Complete the linked documentation/report, Office-preservation, link, PDF/rebuild and focused checks for the proposed publication. Inspect the changed PDFs visually. Resolve findings and verify actual PR/CI/merge and local/remote synchronization using the established workflow; record only observed outcomes in the existing evidence record. Documentation checks do not validate lab controls.
4. Give the owner the five reviewed files and evidence index. In **03 / Owner Acceptance**, have the owner assess analyst report access, engineer maintenance access, unassigned/revoked denial, remote behavior and understandable recovery/evidence records. Record usability observations, limits/follow-ups, reviewer/date and explicit confirmation reference.
5. Keep engineering verdicts, rubric scoring and owner UAT distinct. UAT remains **PENDING** until that confirmation is supplied. Stop at the authorized boundary; do not begin another stage or project task from a publication result.

Check: the public set contains only reviewed safe material, FWT-10 has its actual required evidence, and owner acceptance accurately records the owner's decision. No CI result, score or second tool observation supplies that decision.

If it fails: stop publication on a sensitive-data finding, retain private incident evidence and use the owner's authorized revoke/rotate process if a secret was exposed. History rewriting needs separate approval. For an editorial/build defect, correct the editable source and regenerate; preserve finalized historical reports and raw evidence.

Phase 5 checkpoint and recovery

Use the acceptance criteria: all mandatory tests/subchecks need evidence and PASS for engineering completion; unresolved critical safety issues block acceptance. Owner UAT stays a separate explicit decision. Recover publication errors through reviewed corrections/reverts preserving immutable records; do not rewrite history or expand lab scope to close a report.

Required lab deliverables

These five files are **available as blank templates** in the deliverable library. Work privately, record actual results only after execution and publish only reviewed summaries. The number shorthand used throughout this manual refers to this table.

Filename	Owning record
01-Lab-Environment-and-Architecture.xlsx	Hardware, four VMs, three zones, flows and capacity
02-Identity-and-Zero-Trust-Implementation.docx	Identity, endpoints, HTTPS, app roles and remote configuration
03-Security-Monitoring-and-Testing.xlsx	Events, tests, evidence, automation, recovery and owner acceptance
04-Troubleshooting-and-Lessons.xlsx	Real issues, corrections, retests and lessons
05-Final-Project-Submission.docx	Results summary, rubric, limitations and supporting records

Historical requirement and test transition

The historical mapping appendix preserves the earlier requirement/test labels for readers following old records. It is outside the practical lab instructions and adds no current task or test. Use the current coverage index for the ten FWR/FWT mappings.